



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto *Administrador de BD en consola o terminal*

Curso: Base de Datos II

Docente: Patrick Cuadros Quiroga

Integrantes:

Jahuira Pilco, Dayan Elvis

(2022075749)

Mamani Cori, Cristhian Carlos

(2023077282)

**Tacna – Perú
2026**

Sistema *Administrador de BD en consola o terminal*
Informe de Factibilidad

Versión {1.0}

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	DJ - CM	PCQ	PCQ	3/7/2026	Versión Original

ÍNDICE GENERAL

1. Descripción del Proyecto	4
1.1. Nombre del proyecto	4
1.2. Duración del proyecto	4
1.3. Descripción	4
1.4. Objetivos	5
1.4.1. Objetivo general	5
1.4.2. Objetivos Específicos	5
2. Riesgos.....	5
3. Análisis de la Situación actual.....	5
3.1. Planteamiento del problema.....	5
3.2. Consideraciones de hardware y software.....	6
4. Estudio de Factibilidad.....	6
4.1. Factibilidad Técnica	6
4.2. Factibilidad Económica	7
4.2.1. Costos Generales.....	7
4.2.2. Costos operativos durante el desarrollo.....	7
4.2.3. Costos del ambiente.....	7
4.2.4. Costos de personal.....	7
4.2.5. Costos totales del desarrollo del sistema.....	7
4.3. Factibilidad Operativa.....	7
4.4. Factibilidad Legal	8
4.5. Factibilidad Social	8
4.6. Factibilidad Ambiental	8
5. Análisis Financiero	8
5.1. Justificación de la Inversión	8
5.1.1. Beneficios del Proyecto	8
5.1.2. Criterios de Inversión.....	8
5.1.2.1. Relación Beneficio/Costo (B/C)	8
5.1.2.2. Valor Actual Neto (VAN)	9
5.1.2.3. Tasa Interna de Retorno (TIR).....	9
6. Conclusiones	9

Informe de Factibilidad

1. Descripción del Proyecto

1.1. Nombre del proyecto

Administrador de BD en consola o terminal

1.2. Duración del proyecto

4 meses

1.3. Descripción

El presente proyecto tiene como finalidad el desarrollo de una aplicación de tipo CLI (Command Line Interface) orientada a la administración de bases de datos tanto relacionales como no relacionales. El sistema, denominado NexusDB, permitirá a los usuarios interactuar con gestores de base de datos relacionales (SQLite, PostgreSQL, MySQL) y no relacionales (MongoDB, Redis, Cassandra), mediante comandos estructurados definidos por la aplicación, así como mediante un módulo de generación asistida de consultas SQL a partir de lenguaje natural, apoyado en modelos de lenguaje (IA).

La herramienta será capaz de procesar instrucciones ingresadas por el usuario, interpretarlas mediante un módulo de análisis sintáctico y ejecutarlas sobre la base de datos, permitiendo operaciones de definición y manipulación de datos (DDL y DML) tanto en motores SQL como en sus equivalentes NoSQL. Asimismo, el sistema proporcionará mecanismos básicos de validación, control de errores, auditoría de comandos y visualización de resultados en formato legible dentro del

entorno de consola. El proyecto incluye además un módulo de migración ETL entre distintos motores de base de datos, un panel de rendimiento, un comparador de esquemas, un asistente de voz (NexusVoice) y un módulo de gestión de usuarios con control de permisos.

Este proyecto se orienta tanto al aprendizaje práctico de la administración de bases de datos como al desarrollo de habilidades en el diseño de sistemas interactivos basados en comandos. Adicionalmente, el proyecto contempla la distribución del sistema mediante tres canales complementarios: una extensión para Visual Studio Code publicada en el Marketplace oficial, un servidor MCP (Model Context Protocol) publicado en el repositorio PyPI para su integración con asistentes de inteligencia artificial (Claude, Antigravity, Cursor, entre otros), y un bot de Telegram en modo de solo lectura para consultas remotas seguras. Con ello, se busca que el sistema no sea únicamente una herramienta de uso local, sino un producto de software distribuible y reutilizable por terceros.

1.4. Objetivos

1.4.1. *Objetivo general*

Desarrollar una aplicación en consola que permita administrar una base de datos mediante comandos, facilitando la ejecución de operaciones básicas de gestión de datos.

1.4.2. *Objetivos Específicos*

- Implementar la conexión a una base de datos existente
- Desarrollar un sistema de comandos en consola (CLI)

- Permitir operaciones CRUD sobre las tablas
- Mostrar resultados de manera clara en consola
- Validar comandos y manejar errores básicos
- Incorporar soporte para motores de bases de datos no relacionales (MongoDB, Redis y Cassandra)
- Desarrollar un módulo de migración ETL que permita mover datos entre distintos motores de base de datos
- Integrar un asistente de generación de consultas SQL a partir de lenguaje natural mediante modelos de IA
- Distribuir el sistema como extensión de Visual Studio Code, servidor MCP y bot de Telegram
- Restringir las operaciones expuestas a integraciones externas (MCP y Telegram) a consultas de solo lectura, por motivos de seguridad

2. Riesgos

- Falta de experiencia en conexión a bases de datos
- Problemas de configuración del entorno
- Errores en la interpretación de comandos
- Limitaciones de tiempo para completar todas las funcionalidades
- Fallas en la integración entre módulos
- Incompatibilidad de drivers o versiones entre los distintos motores NoSQL soportados

- Exposición accidental de credenciales de conexión en integraciones externas (bot de Telegram, servidor MCP)
- Indisponibilidad del servidor VPS donde se despliegan el bot de Telegram y las bases de datos de prueba
- Dependencia de servicios de terceros (APIs de modelos de IA, Marketplace de VS Code, PyPI, API de Telegram) que pueden cambiar sus políticas o interfaces

3. Análisis de la Situación actual

3.1. Planteamiento del problema

Las herramientas actuales de administración de bases de datos, en su mayoría, se presentan mediante interfaces gráficas que abstraen el funcionamiento interno de las operaciones, lo que limita la comprensión profunda de los procesos de manipulación de datos.

Por otro lado, en entornos profesionales y de servidores, el uso de interfaces de línea de comandos es predominante debido a su eficiencia, bajo consumo de recursos y capacidad de automatización. Sin embargo, dichas herramientas suelen presentar una curva de aprendizaje elevada.

En este contexto, se identifica la necesidad de desarrollar una solución que permita comprender y aplicar los conceptos de administración de bases de datos mediante una interfaz de consola simplificada y controlada.

Adicionalmente, se observa que la mayoría de herramientas existentes se enfocan exclusivamente en bases de datos relacionales,

dejando de lado a los motores NoSQL, cuyo uso ha crecido de forma sostenida en aplicaciones modernas. Asimismo, la aparición de asistentes de inteligencia artificial capaces de operar herramientas externas mediante protocolos como MCP (Model Context Protocol) representa una oportunidad para modernizar la forma en que los desarrolladores interactúan con sus bases de datos, sin abandonar el control y la seguridad que ofrece una interfaz de consola tradicional.

3.2. Consideraciones de hardware y software

Tipo de Recurso	Nombre	Descripción
Hardware	Computadora personal (PC o laptop)	Intel i5, RAM: 8 GB, HDD: 1 TB, Mouse y Teclado estándar. Equipo para desarrollar y probar el sistema.
Software	Windows 1	Sistema Operativo base para ejecutar herramientas de desarrollo y el sistema.
Software	Python 3.8+	Ampliamente utilizado en aplicaciones CLI; sintaxis clara; gran cantidad de bibliotecas
Software	VS Code	Entornos de desarrollo gratuito con soporte para Python
Software	Node.js + TypeScript	Utilizado para el desarrollo y empaquetado de la extensión de Visual Studio Code
Hardware	Servidor VPS (Contabo, Debian 13)	Utilizado para desplegar de forma permanente el bot de Telegram y las bases de datos de prueba (MySQL, MongoDB, Redis)

4. Estudio de Factibilidad

4.1. Factibilidad Técnica

Cantidad	Recurso	Descripción
1	Laptop	Laptop ASUS, Procesador Ryzen , RAM: 16 GB, SSD: 1 TB y Mouse
1	Laptop	Laptop Lenovo, Procesador Intel Core I5 de 6ta generación, 8 GB de RAM, SSD de 500 GB y Mouse.
1	Servidor VPS	Contabo VPS, Debian 13, usado para desplegar el bot de Telegram como servicio permanente (systemd) y alojar bases de datos de prueba MySQL, MongoDB y Redis.

Conclusión Técnica: El proyecto es técnicamente viable.

Se cuenta con dos equipos con especificaciones adecuadas para el desarrollo, además de un servidor VPS para el despliegue permanente de las integraciones externas. Python es un lenguaje ideal para aplicaciones CLI y no requiere hardware especializado. Las librerías necesarias para la conexión a bases de datos relacionales y NoSQL (psycopg2, mysql-connector-python, pymongo, redis, cassandra-driver) son de código abierto y están disponibles gratuitamente, al igual que las herramientas de publicación utilizadas (PyPI, Visual Studio Code Marketplace y la API de Bots de Telegram).

4.2. Factibilidad Económica

4.2.1. Costos Generales

Item	Cantidad	Costo Unitario (S/.)	Costo Total (S/.)
Laptop para desarrollo	2	2500	5000
Material de escritorio	1	40	40
Total	-	-	5040

4.2.2. Costos operativos durante el desarrollo

Concepto	Costo Mensual (S/.)	Duración (meses)	Costo Total (S/.)
Energía eléctrica	80	4	320
Internet	100	4	400
Hosting VPS (Contabo)	35	4	140
Créditos API de IA (OpenAI/Anthropic)	50	4	200
Total	-	-	1 060

4.2.3. Costos del ambiente

Recurso	Costo Unitario (S/.)	Cantidad	Costo Total (S/.)
Configuración de entorno de desarrollo	50	1	50
Pruebas del sistema	150	1	150
Repositorio GitHub	20	1	20
Pruebas de integración (extensión VSC, servidor MCP, bot de Telegram)	100	1	100

Total	-	-	320
-------	---	---	-----

4.2.4. Costos de personal

Rol	Cantidad	Sueldo Mensual (S/.)	Meses	Subtotal (S/.)
Analista	1	1 500	4	6 000
Desarrollador	1	1 500	4	6 000
Total	-		-	12 000

4.2.5. Costos totales del desarrollo del sistema

Categoría	Total (S/.)
Costos Generales	5 040
Costos Operativos	1 060
Costos del Ambiente	320
Costos de Personal	12 000
TOTAL GENERAL	18 420

4.3. Factibilidad Operativa

Aspecto	Descripción	Estado
Usuarios	Estudiantes de cursos de bases de datos, docentes y desarrolladores que requieran administrar bases de datos desde consola	Viable
Curva de aprendizaje	Baja gracias al comando help y sintaxis intuitiva	Viable
Mantenimiento	El código es simple y modular, fácil de mantener	Viable

Documentación	Se incluirá un manual básico dentro del repositorio	Viable
Soporte	Durante el periodo académico, los desarrolladores brindarán soporte	Viable
Usuarios de integraciones externas	Usuarios de asistentes de IA (Claude, Antigravity, Cursor, entre otros) mediante el servidor MCP, y usuarios de Telegram mediante consultas de solo lectura, sin necesidad de instalar el proyecto completo	Viable

De acuerdo con el análisis presentado en la tabla, la factibilidad operativa del sistema resulta totalmente viable. Los usuarios objetivo tienen el perfil adecuado (conocimiento básico de bases de datos), el sistema incluye un comando de ayuda para facilitar su uso, y los riesgos identificados son controlables mediante una adecuada implementación de manejo de errores y documentación.

4.4. Factibilidad Legal

Aspecto	Descripción	Cumplimiento
Protección de Datos Personales	El sistema no almacena ni procesa datos personales de los usuarios. Solo interactúa con bases de datos locales del usuario.	Sí
Uso de Software	Python es software de código abierto con licencia PSF. Visual Studio Code es gratuito. No se requiere software comercial.	Sí
Propiedad Intelectual	El código desarrollado es propiedad de los autores. Se	Sí

	utilizará licencia MIT para permitir uso académico y comercial.	
Términos de plataformas de distribución	La publicación en el Visual Studio Code Marketplace, PyPI y la API de Bots de Telegram se realiza cumpliendo los acuerdos de publicación de cada plataforma (Marketplace Publisher Agreement, PyPI Terms of Use y Telegram Bot API Terms), sin costo asociado.	Sí

4.5. Factibilidad Social

Aspecto	Descripción	Cumplimiento
Impacto	Permite comprender el funcionamiento interno de bases de datos sin depender de interfaces gráficas	Positivo
Docentes	Herramienta didáctica para enseñar SQL y administración de bases de datos	Positivo
Desarrolladores	Facilita el aprendizaje de conexiones a bases de datos mediante Python	Positivo

4.6. Factibilidad Ambiental

Aspecto	Descripción	Impacto
Uso de papel	No requiere documentación física ni reportes impresos	Positivo
Consumo energético	Funciona en equipos de bajo consumo, sin requerir hardware adicional	Positivo

Residuos electrónicos	Al ser software puro, no genera residuos físicos	Positivo
-----------------------	--	----------

5. Análisis Financiero

5.1. Justificación de la Inversión

La inversión en el desarrollo del Administrador de BD en consola se justifica por los siguientes motivos:

- Eliminación de dependencia de herramientas gráficas comerciales como DBeaver Pro o Navicat
- Reducción del tiempo de aprendizaje para comandos SQL mediante una interfaz simplificada
- Automatización de tareas repetitivas de administración de bases de datos
- Disponibilidad de una herramienta didáctica gratuita para la enseñanza de bases de datos
- Cero costos en infraestructura por ser una aplicación 100% Python
- Ampliación del público alcanzable mediante distribución en el Marketplace de VS Code, PyPI y Telegram, sin costos de licenciamiento
- Reducción de la barrera de entrada al uso de bases de datos NoSQL mediante una sintaxis unificada con el motor relacional

5.1.1. Beneficios del Proyecto

Beneficios Intangibles

- Fortalecimiento de competencias técnicas en el equipo desarrollador
- Contribución al aprendizaje práctico de bases de datos
- Disponibilidad de código fuente para futuras adaptaciones
- Independencia de plataformas comerciales
- Código ligero y portable al usar solo Python estándar
- Experiencia práctica del equipo en integración con protocolos emergentes de IA (MCP) y en buenas prácticas de seguridad para bots conversacionales
- Portafolio de distribución real (Marketplace, PyPI y Telegram) que fortalece el perfil profesional de los integrantes

5.1.2. Criterios de Inversión

5.1.2.1. Relación Beneficio/Costo (B/C)

Año	Beneficios S/.	Mantenimiento S/.	Beneficio Neto
0	0	18 420	-18 420
1	8 500	500	8000
2	9 500	500	9000
3	11 000	500	10 500

$B/C = \text{Beneficios netos actualizados} / \text{Inversión inicial}$

$$B/C = (8,000/1.12 + 9,000/1.12^2 + 10,500/1.12^3) / 18,420$$

$$B/C = 21,793.27 / 18,420 = 1.18$$

Interpretación: Como B/C es mayor a 1, por cada sol invertido el proyecto genera S/. 1.18 en beneficios actualizados, lo que indica que el proyecto es rentable.

5.1.2.2. *Valor Actual Neto (VAN)*

$$\text{VAN} = -18,420 + 8,000 / (1.12)^1 + 9,000 / (1.12)^2 + 10,500 / (1.12)^3$$

$$\text{VAN} = -18,420 + 7,142.86 + 7,175.51 + 7,474.90$$

$$\text{VAN} = \text{S/. } 3,373.27$$

Interpretación: VAN es mayor a 0, por lo tanto, el proyecto genera valor por encima de lo necesario para recuperar la inversión, considerando una tasa de descuento del 12%.

5.1.2.3. *Tasa Interna de Retorno (TIR)*

La TIR se calcula como la tasa que hace el VAN igual a cero:

$$\text{VAN} = 0 = -11,370 + 4,100 / (1+\text{TIR})^1 + 4,550 / (1+\text{TIR})^2 + 5,000 / (1+\text{TIR})^3$$

Resolviendo la ecuación mediante interpolación:

Probando con tasa 21 por ciento:

$$\text{VAN} = -18,420 + 8,000/1.21 + 9,000/1.21^2 + 10,500/1.21^3$$

$$\text{VAN} = -18,420 + 6,611.57 + 6,147.12 + 5,926.62 = 265.31$$

Probando con tasa 22 por ciento:

$$\text{VAN} = -18,420 + 8,000/1.22 + 9,000/1.22^2 + 10,500/1.22^3$$

$$\text{VAN} = -18,420 + 6,557.38 + 6,047.03 + 5,783.36 = -32.23$$

$$\text{TIR} \approx 21.9 \text{ por ciento}$$

6. Conclusiones

El análisis de factibilidad realizado para el proyecto Administrador de BD en consola o terminal arroja los siguientes resultados:

Factibilidad Técnica: *El proyecto es viable pues se cuenta con los conocimientos y herramientas necesarias para su desarrollo. Python con sus librerías estándar y de terceros permite construir una aplicación CLI completa con soporte SQL y NoSQL, integraciones de IA y distribución multicanal, sin requerir infraestructura costosa adicional más allá de un servidor VPS de bajo costo.*

Factibilidad Económica: *La inversión total asciende a S/. 18,420.00. Los indicadores financieros muestran resultados favorables con un VAN de S/. 3,373.27, una TIR de 21.9 por ciento y una relación Beneficio/Costo de 1.18, todos superiores a los criterios mínimos establecidos (VAN mayor a 0 y TIR mayor a la tasa de descuento del 12 por ciento).*

Factibilidad Operativa: La interfaz de línea de comandos con comando help facilita la curva de aprendizaje. Los usuarios objetivo estudiantes y docentes cuentan con el perfil adecuado. Adicionalmente, la distribución mediante extensión de VS Code, servidor MCP y bot de Telegram amplía el alcance operativo a usuarios que no necesariamente usan la terminal como interfaz principal.

Factibilidad Legal: El proyecto utiliza exclusivamente software de código abierto con licencias permisivas, cumpliendo con las normativas de propiedad intelectual.

Factibilidad Social: El impacto es positivo al contribuir con la formación de los estudiantes y ofrecer una herramienta didáctica para la enseñanza de bases de datos.

Factibilidad Ambiental: El proyecto no genera residuos electrónicos ni consume recursos adicionales, promoviendo el uso de software libre.